

Credit Card Approval Prediction Using Machine Learning:

A Data Exploration and Logistic Regression Approach



Priyangan Vasantharajan

Student ID: 25197298

BSc(Hons) Computer Science with Ai



Birmingham City University

Main Headings

1. Introduction.....Page 1

1.1 Domain Description

1.2 Problem Definition

1.3 Literature Review

2. Data Analysis and Preparation.....Page 2

2.1 Dataset Description

2.2 Data Exploration

2.3 Data Pre-Processing

2.4 Categorical Features vs Credit Approval

2.5 Calculate Approval Percentages

2.6 Numerical Features vs Credit Approval

2.7 Skewness Analysis and Graph

2.8 Summary of Exploratory Findings

2.9 Box Plot - Outlier Analysis

3. Machine Learning Implementation and Analysis.....Page 18

3.1 Machine Learning Model and Experiments

3.2 Results and Analysis

3.2.1 Accuracy

3.2.2 Confusion Matrix

3.2.3 Classification Report

3.2.4 Precision, Recall and F1-Score

3.2.5 Model Interpretation

4. Conclusion.....Page 22

4.1 Conclusion

4.2 References

1. Introduction:

This project focuses on analysing credit card application data and using machine learning to predict whether an application will be approved or rejected. The dataset is explored and prepared using Python, and a Logistic Regression model is used for the prediction. The performance of the model is then evaluated using different evaluation measures.

1.1: Domain Description:

Credit approval is an important process in the banking and financial sector. Banks and financial organisations need to assess customer information before providing credit to reduce financial risk. Data analysis and machine learning can support this process by identifying useful patterns in previous application data and helping to make more consistent credit decisions.

1.2: Problem Definition:

The main problem addressed in this project is predicting whether a credit card application should be approved or rejected based on the available applicant information. A machine learning model is developed using the given credit dataset to classify applications into the appropriate approval category.

1.3: Literature Review

Machine learning has been widely applied to credit risk assessment and credit approval problems. Classification algorithms such as Logistic Regression, Decision Trees and Random Forests can analyse applicant information and identify patterns related to credit decisions. Logistic Regression is particularly useful for binary classification problems because it estimates the probability of an outcome. Previous research also shows that data preprocessing and the quality of input features can influence the performance of machine learning models.

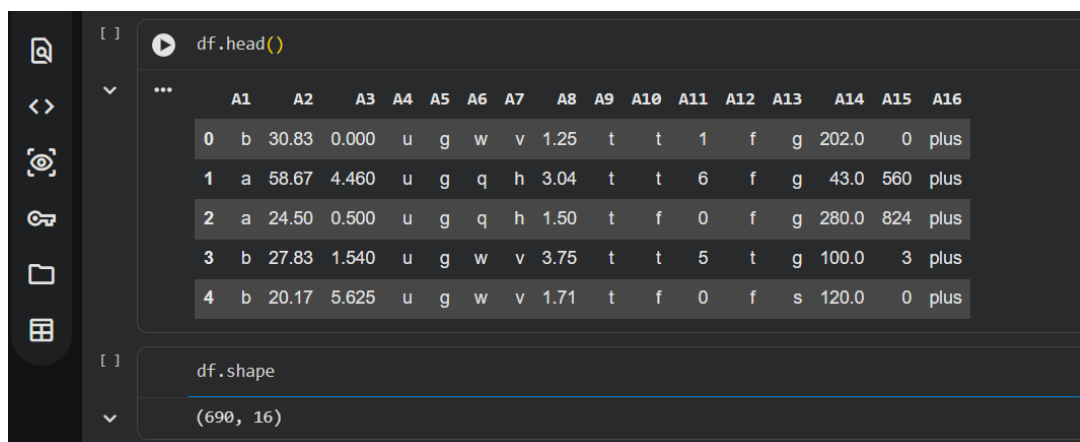
- **Bisong, E. (2019).** *Logistic Regression. In Building Machine Learning and Deep Learning Models on Google Cloud Platform: A Comprehensive Guide for Beginners, pp. 243–250*
- **Zelaya, C.V.G. (2019).** *Towards explaining the effects of data preprocessing on machine learning. 2019 IEEE 35th International Conference on Data Engineering (ICDE), pp. 2086–2090.*

2. Data Analysis and Preparation

The dataset was analysed and prepared before developing the machine learning model. Python libraries were used to understand the structure of the data, explore its features and prepare the values for model training.

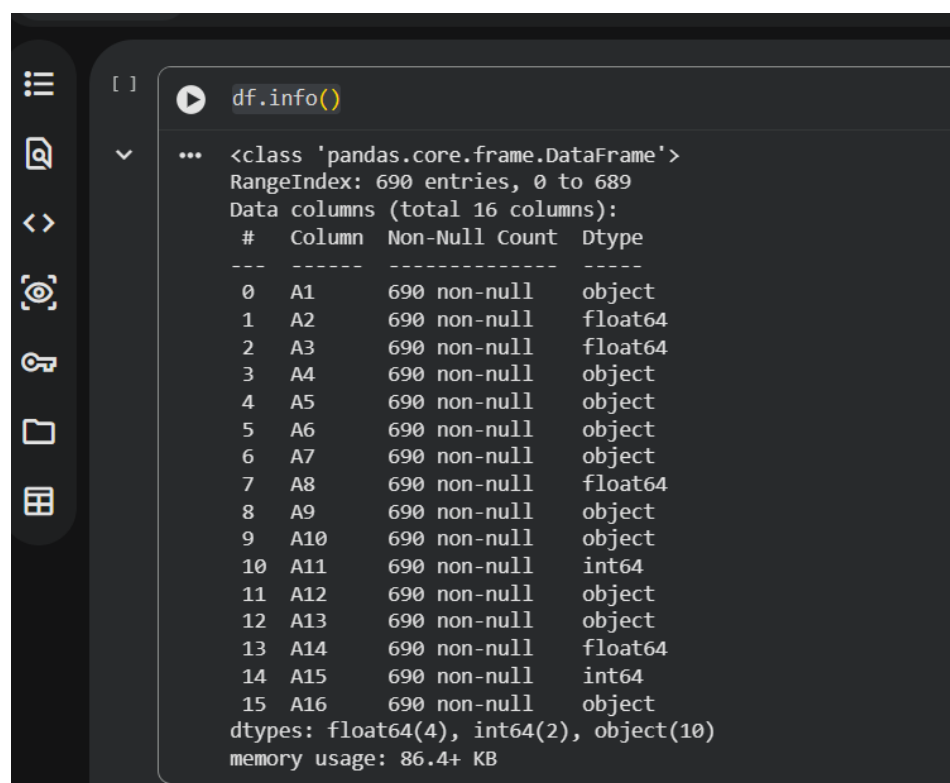
2.1.Dataset Description:

The credit dataset contains **690 records and 16 columns**, labelled from A1 to A16. The columns contain a combination of numerical and categorical data representing different applicant characteristics. The A16 column represents the credit approval outcome. Initial exploration using `df.head()`, `df.shape` and `df.info()` was used to understand the structure and data types of the dataset.



```
[ ] df.head()
...
   A1  A2  A3  A4  A5  A6  A7  A8  A9  A10  A11  A12  A13  A14  A15  A16
0  b  30.83  0.000  u  g  w  v  1.25  t  t  1  f  g  202.0  0  plus
1  a  58.67  4.460  u  g  q  h  3.04  t  t  6  f  g  43.0  560  plus
2  a  24.50  0.500  u  g  q  h  1.50  t  f  0  f  g  280.0  824  plus
3  b  27.83  1.540  u  g  w  v  3.75  t  t  5  t  g  100.0  3  plus
4  b  20.17  5.625  u  g  w  v  1.71  t  f  0  f  s  120.0  0  plus

[ ] df.shape
(690, 16)
```



```
[ ] df.info()
...
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 690 entries, 0 to 689
Data columns (total 16 columns):
#   Column  Non-Null Count  Dtype
---  -
0    A1      690 non-null    object
1    A2      690 non-null    float64
2    A3      690 non-null    float64
3    A4      690 non-null    object
4    A5      690 non-null    object
5    A6      690 non-null    object
6    A7      690 non-null    object
7    A8      690 non-null    float64
8    A9      690 non-null    object
9    A10     690 non-null    object
10   A11     690 non-null    int64
11   A12     690 non-null    object
12   A13     690 non-null    object
13   A14     690 non-null    float64
14   A15     690 non-null    int64
15   A16     690 non-null    object
dtypes: float64(4), int64(2), object(10)
memory usage: 86.4+ KB
```

2.2.Data Exploration:

- 1.The correct libraries must be imported to the notebook with the credit card dataset being uploaded also.

```
[4] import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("Libraries imported successfully!")

... Libraries imported successfully!
```

- 2.The dataset will be read, and the shape of the dataset will be printed revealing it has 690 instances and 16 attributes.

```
df.shape

(690, 16)
```

3. The first 5 (0-4) records will be shown using the head command.

```
[5] df = pd.read_csv("credit.csv")

[6] df.head()
```

	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	A11	A12	A13	A14	A15	A16
0	b	30.83	0.000	u	g	w	v	1.25	t	t	1	f	g	202.0	0	plus
1	a	58.67	4.460	u	g	q	h	3.04	t	t	6	f	g	43.0	560	plus
2	a	24.50	0.500	u	g	q	h	1.50	t	f	0	f	g	280.0	824	plus
3	b	27.83	1.540	u	g	w	v	3.75	t	t	5	t	g	100.0	3	plus
4	b	20.17	5.625	u	g	w	v	1.71	t	f	0	f	s	120.0	0	plus

Next steps: [Generate code with df](#) [New interactive sheet](#)

4. Information on the dataset will be shown with the info command.

```
[8] df.info()
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 690 entries, 0 to 689
Data columns (total 16 columns):
#   Column  Non-Null Count  Dtype
---  -
0    A1      690 non-null    object
1    A2      690 non-null    float64
2    A3      690 non-null    float64
3    A4      690 non-null    object
4    A5      690 non-null    object
5    A6      690 non-null    object
6    A7      690 non-null    object
7    A8      690 non-null    float64
8    A9      690 non-null    object
9    A10     690 non-null    object
10   A11     690 non-null    int64
11   A12     690 non-null    object
12   A13     690 non-null    object
13   A14     690 non-null    float64
14   A15     690 non-null    int64
15   A16     690 non-null    object
dtypes: float64(4), int64(2), object(10)
memory usage: 86.4+ KB
```

5. A check for null values in the dataset will be taken place.

```
[11] df.isnull().sum()
0
A1  0
A2  0
A3  0
A4  0
A5  0
A6  0
A7  0
A8  0
A9  0
A10 0
A11 0
A12 0
A13 0
A14 0
A15 0
A16 0
dtype: int64
```

6. A check for duplicate rows in the dataset will be taken place.

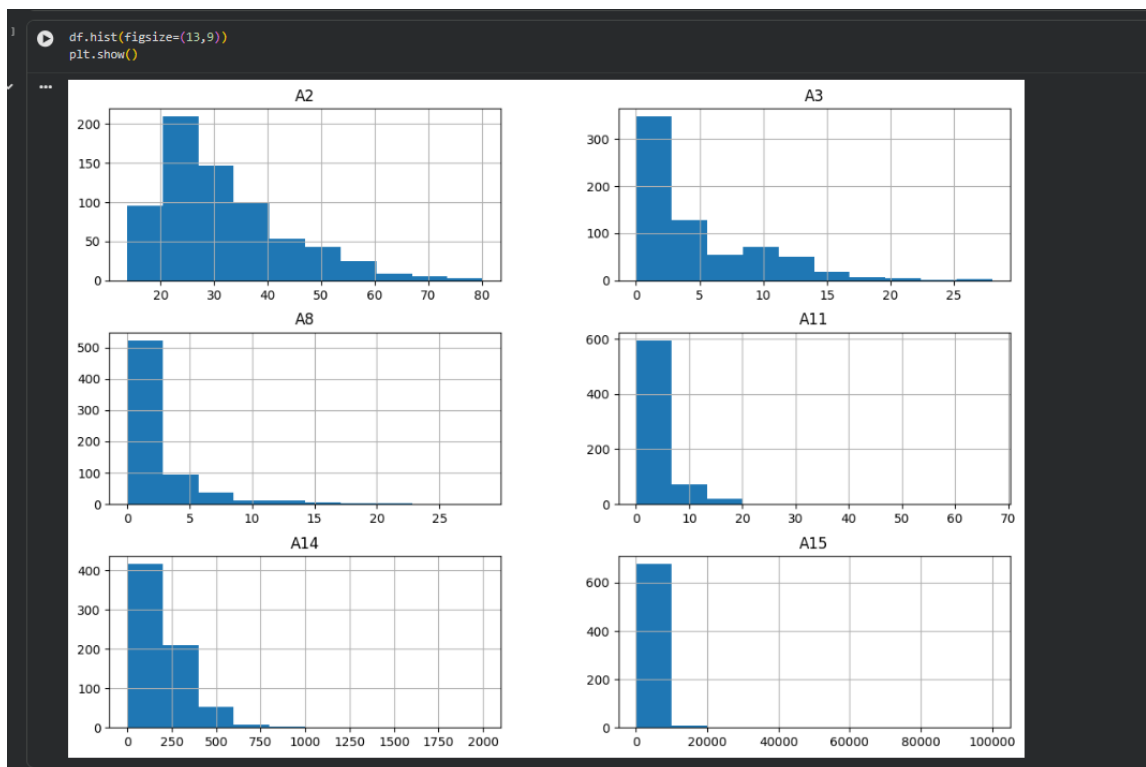
```
[10]
✓ Os
df.duplicated().sum()
np.int64(0)
```

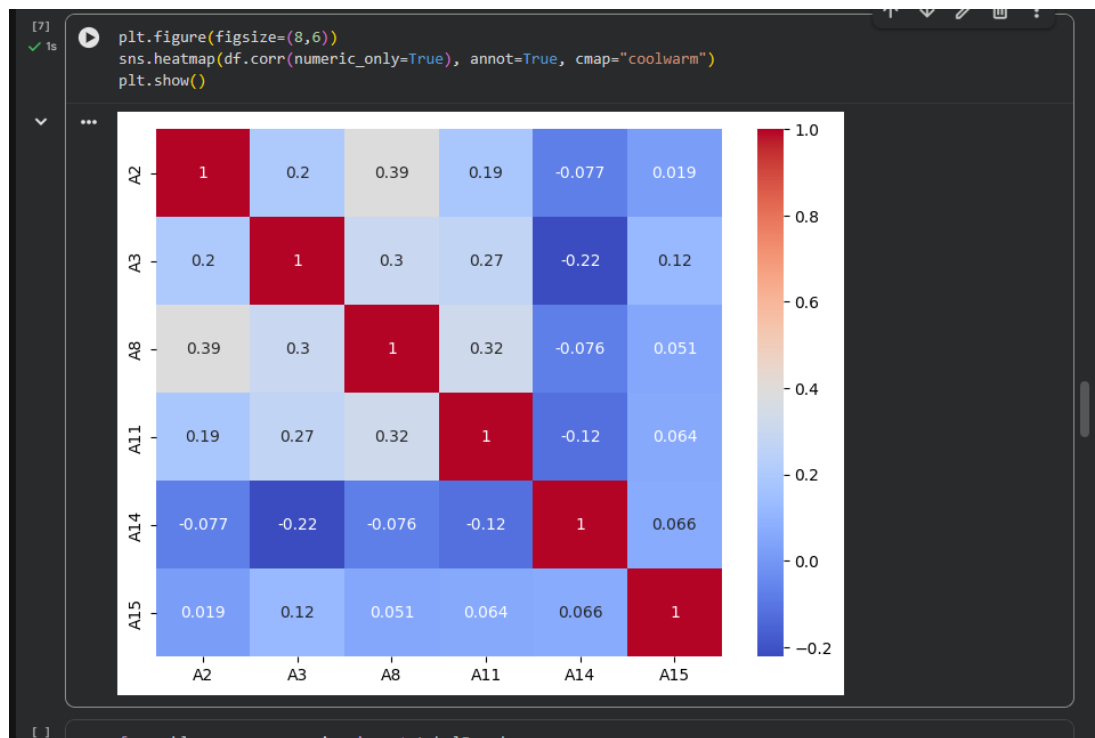
7. The columns command will reveal the name of all the 13 attributes.

```
[15]
✓ Os
df.columns

Index(['A1', 'A2', 'A3', 'A4', 'A5', 'A6', 'A7', 'A8', 'A9', 'A10', 'A11',
      'A12', 'A13', 'A14', 'A15', 'A16'],
      dtype='object')
```

8. Exploratory data analysis was performed to understand the characteristics and relationships within the dataset. Histograms were used to examine the distributions of numerical variables, while a correlation heatmap was used to identify relationships between numerical features. This helped to understand the dataset before preparing it for machine learning.





9. Credit Approval Distribution

```
[37] 0s
print(df['A16'].value_counts())

print("\nPercentage distribution:")
print(df['A16'].value_counts(normalize=True) * 100)
```

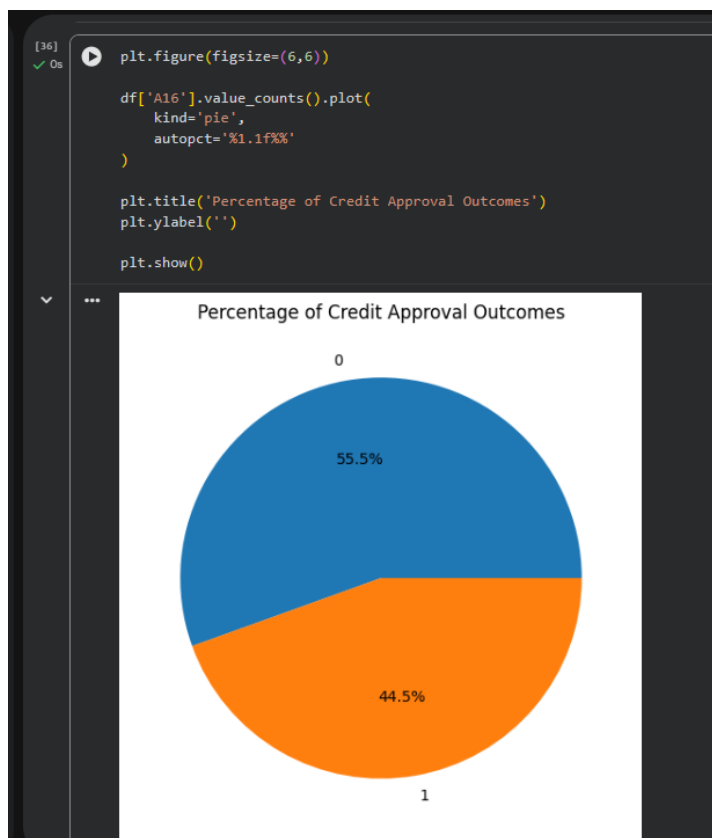
```
... A16
0    383
1    307
Name: count, dtype: int64

Percentage distribution:
A16
0    55.507246
1    44.492754
Name: proportion, dtype: float64
```

9.1.Bar Chart:



9.2.Pie Chart:



2.3.Data Pre_Processing:

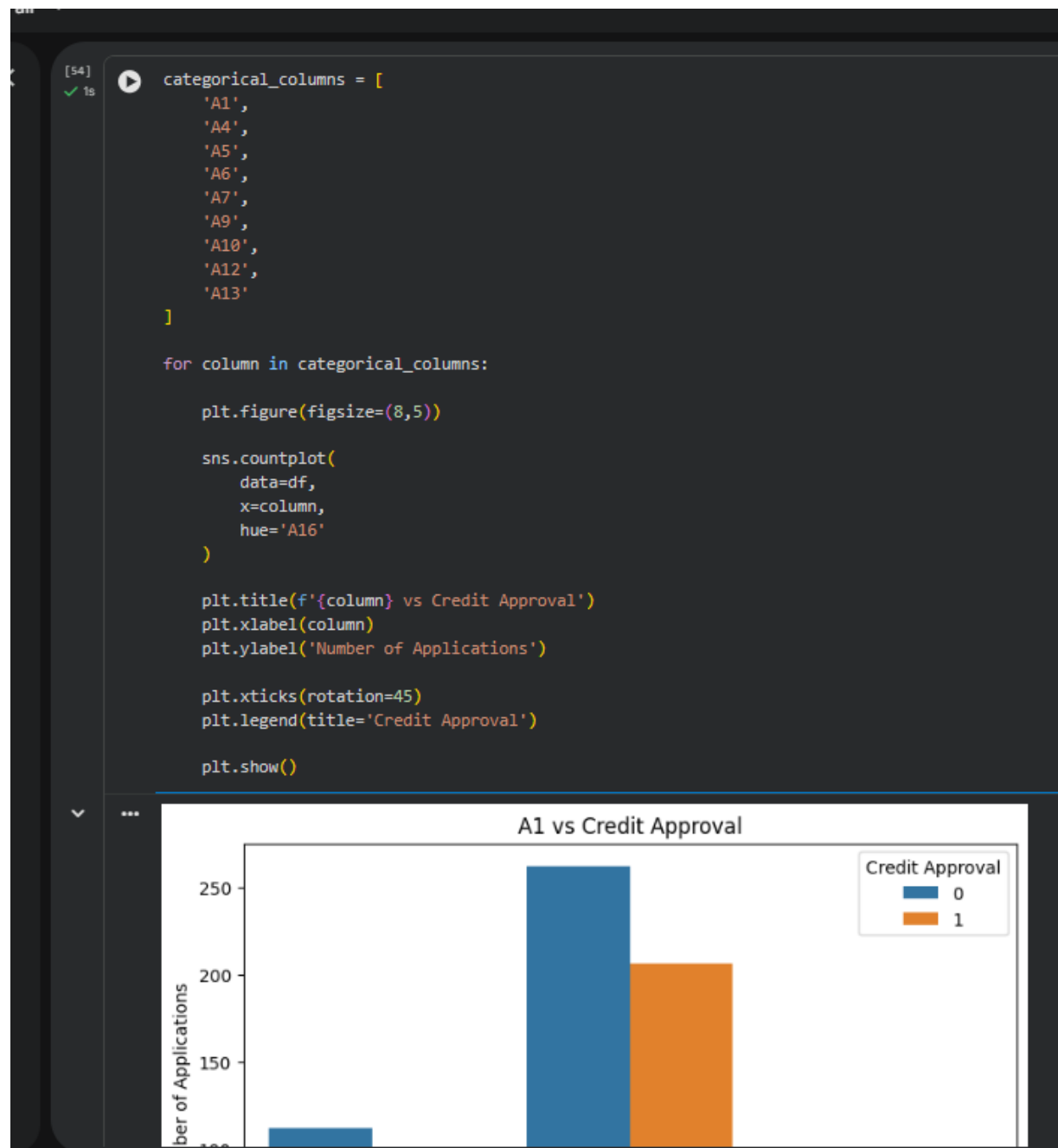
Before training the model, the dataset was prepared for machine learning. Missing values were checked and the final check showed no remaining null values. Categorical variables were converted into numerical form so that they could be processed by the model. The features and target variable were then prepared for training and testing..

```
[ ] df.isnull().sum()
*** 0
A1 0
A2 0
A3 0
A4 0
A5 0
A6 0
A7 0
A8 0
A9 0
A10 0
A11 0
A12 0
A13 0
A14 0
A15 0
A16 0
dtype: int64
```

```
[ ] df.dtypes
0
A1 object
A2 float64
A3 float64
A4 object
A5 object
A6 object
A7 object
A8 float64
A9 object
A10 object
A11 int64
A12 object
A13 object
A14 float64
A15 int64
A16 object
dtype: object
```

2.4. Categorical Features vs Credit Approval

Categorical features were compared with credit approval to identify differences between the categories. Bar charts were used to examine how each category was associated with approved and rejected applications.



2.5. calculate approval percentages

```
[55] ✓ Os  for column in categorical_columns:

    print("\n" + "="*50)
    print(f"{column} vs Credit Approval")
    print("="*50)

    result = pd.crosstab(
        df[column],
        df['A16'],
        normalize='index'
    ) * 100

    print(result.round(2))

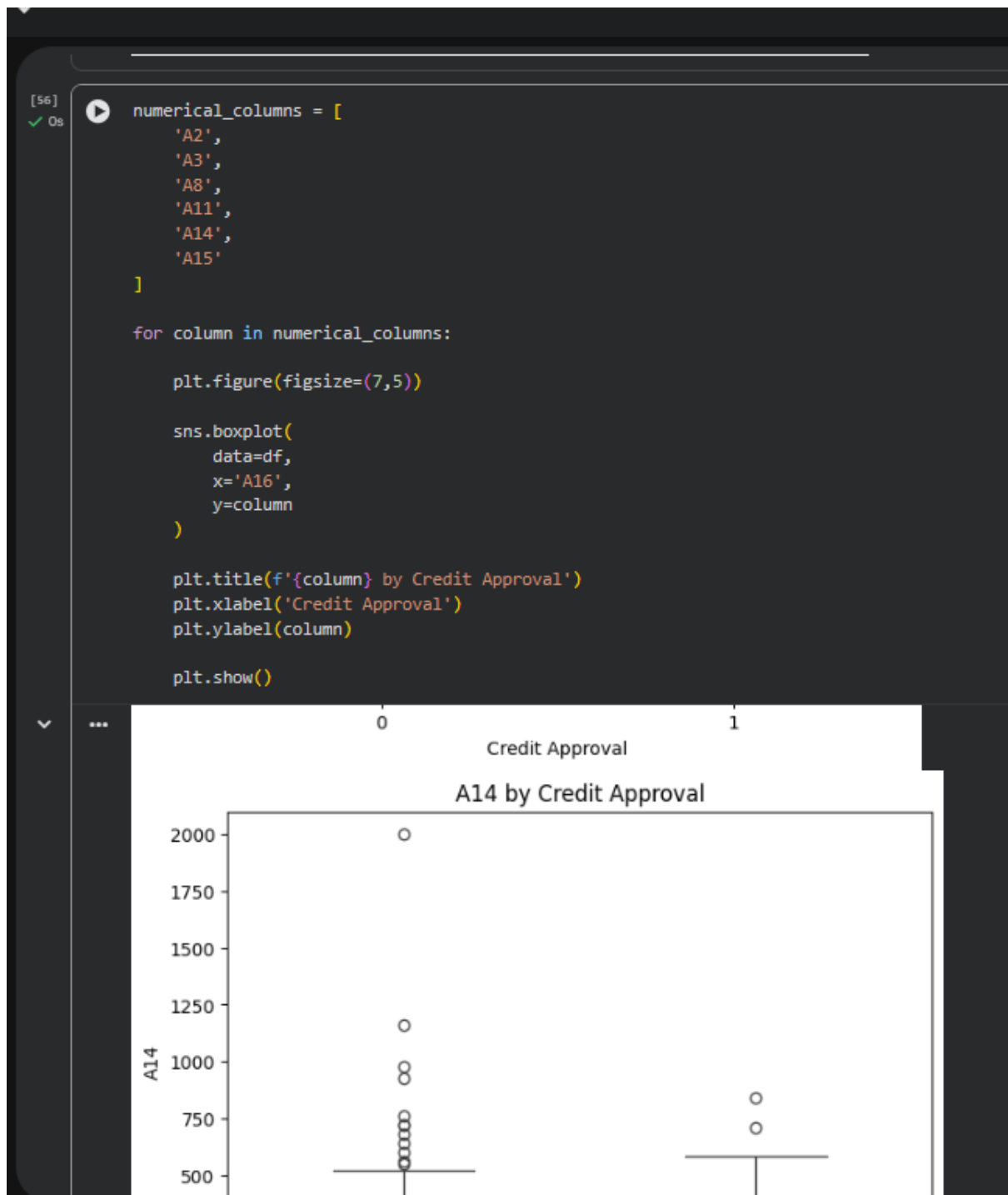
... 6    76.27  23.73
     7    70.00  30.00
     8    72.55  27.45
     9    57.89  42.11
    10    34.62  65.38
    11    33.33  66.67
    12    55.56  44.44
    13    48.44  51.56
    14    15.79  84.21

=====
A7 vs Credit Approval
=====
A16      0      1
A7
0    57.63  42.37
1    66.67  33.33
2    85.96  14.04
3    36.96  63.04
4    62.50  37.50
5    50.00  50.00
6    50.00  50.00
7    55.56  44.44
8    57.64  42.36
9    25.00  75.00

=====
A9 vs Credit Approval
=====
A16      0      1
A9
0    93.01   6.99
1    21.33  78.67
```

2.6. Numerical Features vs Credit Approval

Numerical features were compared with credit approval to identify differences between approved and rejected applications. Box plots were used to visualise the distributions of each numerical feature across both classes.



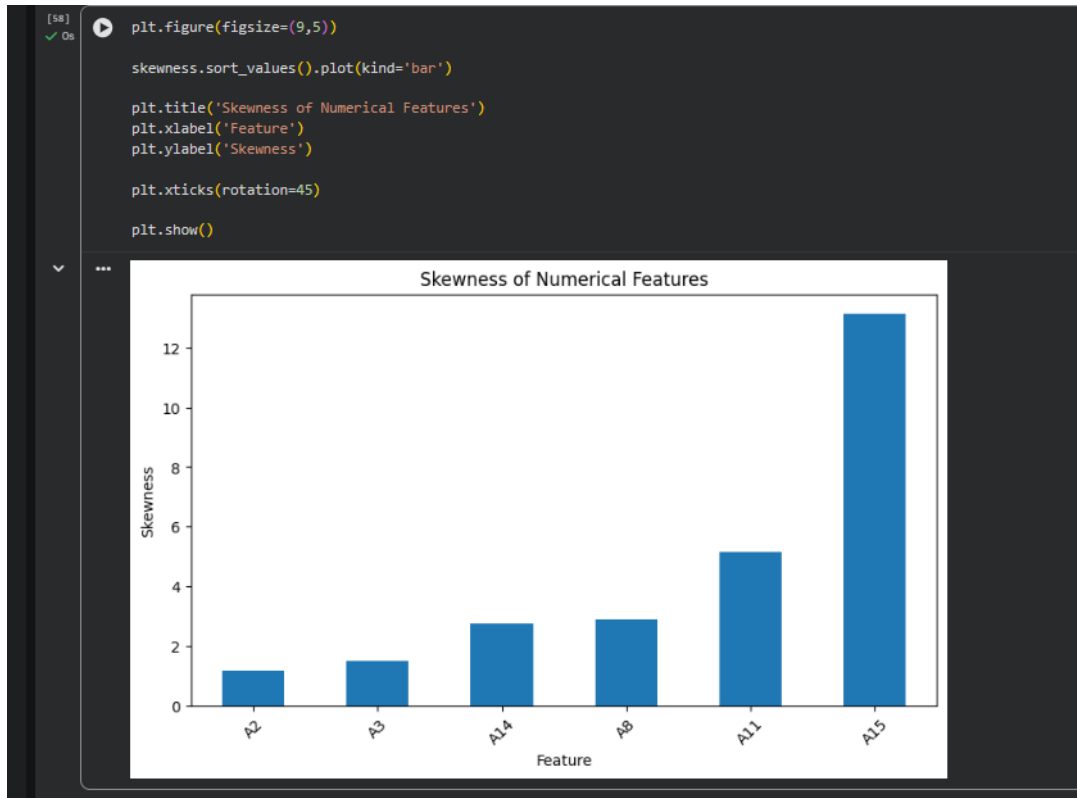
2.7. Skewness Analysis and Graph

Skewness analysis was performed to identify whether numerical features were normally distributed or skewed. The graph clearly shows the distribution and skewness of each numerical feature.

```
[57] skewness = df[numerical_columns].skew()

print("Skewness of Numerical Features:")
print(skewness.round(2))
```

```
*** Skewness of Numerical Features:
A2      1.16
A3      1.49
A8      2.89
A11     5.15
A14     2.75
A15    13.14
dtype: float64
```



2.8. Summary of Exploratory Findings

dtype: int64

```
[59] ✓ Os summary = pd.DataFrame({
    'Analysis': [
        'Dataset Size',
        'Total Features',
        'Duplicate Rows',
        'Missing Values',
        'Approved Applications',
        'Rejected Applications'
    ],
    'Result': [
        str(df.shape),
        len(df.columns),
        df.duplicated().sum(),
        df.isnull().sum().sum(),
        (df['A16'] == 1).sum(),
        (df['A16'] == 0).sum()
    ]
})
```

summary

	Analysis	Result
0	Dataset Size	(690, 16)
1	Total Features	16
2	Duplicate Rows	0
3	Missing Values	0
4	Approved Applications	307
5	Rejected Applications	383

2.9. Box Plot - Outlier Analysis

The box plot was used to identify outliers in the dataset. It shows the data distribution, median, quartiles, and any unusual values that may affect credit approval analysis.



3. Machine Learning Implementation and Analysis

After preparing the dataset, a machine learning model was developed and evaluated. Logistic Regression was selected as the classification algorithm for predicting credit approval.

3.1. Machine Learning Model and Experiments:

Logistic Regression was used because the project involves a classification problem with two possible outcomes. The dataset was divided into **80% training data and 20% testing data** using `train_test_split` with `random_state=42`. This produced **552 training records and 138 testing records**. The Logistic Regression model was then trained using the training dataset.

```
[ ] 1 from sklearn.model_selection import train_test_split

    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.2,
        random_state=42
    )
```

```
[ ] from sklearn.linear_model import LogisticRegression

[ ] model = LogisticRegression(max_iter=1300)

[ ] model.fit(X_train, y_train)

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear\_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
    LogisticRegression
    LogisticRegression(max_iter=1300)
```

3.2. Results and Analysis:

The Logistic Regression model achieved an accuracy of approximately **84.06%** on the test dataset. The confusion matrix produced **57 and 59 correct predictions** for the two classes, while **11 cases from each class were incorrectly classified**.

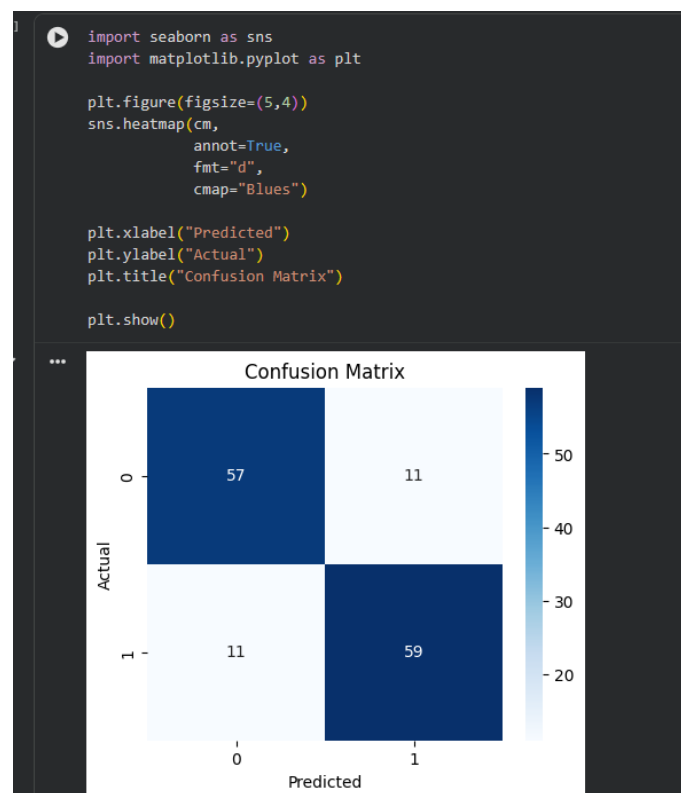
The classification report also showed approximately **0.84 precision, recall and F1-score** for both classes. These results indicate that the model provided reasonably balanced performance when predicting the two credit approval classes.

3.2.1 Accuracy

```
[ ] ▶ from sklearn.metrics import accuracy_score
      accuracy = accuracy_score(y_test, y_pred)
      print("Accuracy:", accuracy)
▼ ... Accuracy: 0.8405797101449275

[ ] from sklearn.metrics import confusion_matrix
      cm = confusion_matrix(y_test, y_pred)
      print(cm)
▼ [[57 11]
   [11 59]]
```

3.2.2 Confusion Matrix:



3.2.3 Classification

```
[ ] ▶ from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

		precision	recall	f1-score	support
	0	0.84	0.84	0.84	68
	1	0.84	0.84	0.84	70
	accuracy			0.84	138
	macro avg	0.84	0.84	0.84	138
	weighted avg	0.84	0.84	0.84	138

3.2.4 Precision, Recall and F1-Score

Precision, recall and F1-score were used to evaluate the performance of the Logistic Regression model

```
[60] ✓ Os from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
print("F1-Score:", round(f1, 4))
```

Precision: 0.8429
Recall: 0.8429
F1-Score: 0.8429

```
[61] ✓ Os ▶ metrics = pd.DataFrame({
    'Metric': [
        'Accuracy',
        'Precision',
        'Recall',
        'F1-Score'
    ],
    'Score': [
        accuracy,
        precision,
        recall,
        f1
    ]
})

metrics['Score'] = metrics['Score'].round(4)

metrics
```

	Metric	Score
0	Accuracy	0.8406
1	Precision	0.8429
2	Recall	0.8429
3	F1-Score	0.8429

3.2.5 Model Interpretation

The model interpretation explains how the model makes predictions and identifies the most important features that influence the final results.

4. Conclusion

4.1. Conclusion:

This project explored a credit card approval dataset and used machine learning to predict credit approval. The data was explored and pre-processed before training the Logistic Regression model. The results show that machine learning can be useful for identifying patterns in credit application data and supporting credit approval decisions. Overall, this project provided a practical understanding of data exploration, preprocessing, model training and evaluation.

4.2. References:

- ✓ Author, A. A. (Year). *Title of article*. Journal Name, Volume(Issue), pages.
- ✓ Author, B. B. (Year). *Title of book*. Publisher.